AGORA — Phase 1 Implementation Prompt

Week 1: Core Engines (Dave's Tasks)

Context: You are building **Agora**, an on-chain multi-agent orchestration primitive that intelligently selects whether AI agents should debate, vote, or use alternative consensus mechanisms to resolve a task. It executes the selected mechanism with full transparency, builds Merkle trees of all deliberation artifacts, stores cryptographic receipts on Solana, and releases payment trustlessly on quorum. See the attached **Synthesized Architecture Report** for full design context.

Your role: You are implementing Dave's Week 1 deliverables — the ML/LLM core of the system. Josh is handling the Anchor smart contract and FastAPI layer separately. Joshua Ddf is handling the React dashboard.

Code standards:

- Python 3.11+, fully typed with type hints on all function signatures
- Use `pydantic` BaseModel for all data structures (not dataclasses)
- Async where appropriate (LLM calls, I/O)
- Google-style docstrings on all public functions/classes
- Production-ready: proper error handling, logging via `structlog`, no bare `except` clauses
- All LLM calls go through a unified `AgentCaller` abstraction (defined below) so we can swap backends

LLM SDK: We use the **Vertex AI SDK** (`google-cloud-aiplatform` + `langchain-google-vertexai`) for all model calls. This gives us access to both Gemini and Claude models under a single API. The model routing strategy is:

- `flash_caller()` → `gemini-2.0-flash` — cost-efficient generation for opening statements, voting, rebuttals
- `pro_caller()` → `gemini-2.5-pro` — strong reasoning for cross-examination, mechanism selection, final synthesis
- `claude_caller()` → `claude-sonnet-4-6` (via Vertex AI Model Garden) — alternative reasoning model for diversity or fallback
- All models are accessed via LangChain's `ChatVertexAI` (for Gemini) or `ChatAnthropicVertex` (for Claude on Vertex)
- Requires: `GOOGLE_CLOUD_PROJECT` env var set, Vertex AI API enabled, and for Claude models, the Anthropic model garden API enabled

---

Task 0: Repository Structure

Create the monorepo structure. All code lives under `agora/`.

Package manager: Use `uv` as the preferred package manager. Set up the project environment before anything else:

```
# Initialize the project with uv
cd agora/
uv venv agora-env --python 3.11
source agora-env/bin/activate  # Linux/Mac

# After creating pyproject.toml (below), install all dependencies:
uv pip install -e ".[dev]"

# To add new dependencies later:
```

```
uv pip install <package>
# Then add to pyproject.toml manually to keep it as source of truth
```

Monorepo structure:

```
agora/
├── README.md
├── pyproject.toml                    # Single Python package config (uv-compatible)
├── .python-version                   # Pin to 3.11 for consistency
├── .github/
│   └── workflows/
│       └── ci.yml
├── agora/                            # Main Python package
│   ├── __init__.py
│   ├── config.py                     # Global config (GCP project, model names, thresholds)
│   ├── types.py                      # All shared Pydantic models
│   ├── agent.py                      # AgentCaller abstraction
│   ├── selector/                     # Mechanism Selector
│   │   ├── __init__.py
│   │   ├── bandit.py                 # Thompson Sampling contextual bandit
│   │   ├── reasoning.py              # LLM reasoning agent wrapper
│   │   ├── features.py               # Task feature extraction
│   │   └── selector.py               # Combined selector (bandit + reasoning)
│   ├── engines/                      # Deliberation Engines
│   │   ├── __init__.py
│   │   ├── debate.py                 # Factional Debate Engine (LangGraph)
│   │   ├── vote.py                   # ISP-Weighted Vote Engine (LangGraph)
│   │   ├── delphi.py                 # (Phase 3 extension — stub only for now)
│   │   └── moa.py                    # (Phase 3 extension — stub only for now)
│   ├── runtime/                      # Orchestration layer
│   │   ├── __init__.py
│   │   ├── orchestrator.py           # Main orchestration pipeline
│   │   ├── monitor.py                # Mid-execution state monitor
│   │   └── hasher.py                 # Transcript hasher + Merkle tree builder
│   ├── solana/                       # Solana integration (Josh fills in, Dave stubs)
│   │   ├── __init__.py
│   │   └── client.py                 # Stub: interface for on-chain operations
│   └── sdk/                          # Public SDK (Phase 2)
│       ├── __init__.py
│       └── arbitrator.py             # Stub for now
├── contract/                         # Anchor Smart Contract (Josh's domain)
│   └── ...
├── api/                              # FastAPI Bridge (Josh's domain)
│   └── ...
├── frontend/                         # React Dashboard (Joshua Ddf's domain)
│   └── ...
├── benchmarks/                       # Evaluation suite
│   ├── __init__.py
│   ├── runner.py
│   └── datasets/
└── tests/
    ├── test_bandit.py
    ├── test_debate.py
    ├── test_vote.py
    ├── test_hasher.py
    └── test_orchestrator.py
```

Set up `pyproject.toml` with these dependencies:

```toml
[project]
name = "agora-protocol"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "langgraph>=0.3.0",
    "langchain-google-vertexai>=2.0",
    "langchain-core>=0.3.0",
    "google-cloud-aiplatform>=1.70",
    "pydantic>=2.0",
    "structlog>=24.0",
    "numpy>=1.26",
    "merkletools>=1.0.3",
    "httpx>=0.27",
]

[project.optional-dependencies]
dev = ["pytest>=8.0", "pytest-asyncio>=0.24", "ruff>=0.8"]
```

Task 1: Shared Types (`agora/types.py`)

Define ALL shared data models upfront. Every other module imports from here. This is the source of truth for data flowing through the system.

```
"""
Define the following Pydantic models:

1. MechanismType — str enum: "debate", "vote", "delphi", "moa"

2. TaskFeatures — extracted features used by the mechanism selector:
   - task_text: str                          # Raw task/question text
   - complexity_score: float                 # 0.0-1.0, computed from token count + clause
nesting
   - topic_category: str                     # e.g., "math", "reasoning", "factual",
"creative", "code"
   - expected_disagreement: float            # 0.0-1.0, estimated from topic type
   - answer_space_size: int                  # Estimated number of distinct valid answers
(1=factual, many=creative)
   - time_sensitivity: float                 # 0.0-1.0, from task metadata
   - agent_count: int                        # Number of agents participating
   - stakes: float                           # 0.0-1.0, payment amount normalized

3. MechanismSelection — output of the selector:
   - mechanism: MechanismType
   - confidence: float                       # 0.0-1.0
   - reasoning: str                          # LLM chain-of-thought explanation
   - reasoning_hash: str                     # SHA-256 of the reasoning string
   - bandit_recommendation: MechanismType    # What the bandit alone suggested
   - bandit_confidence: float                # Bandit's confidence

4. AgentOutput — a single agent's contribution in any engine:
   - agent_id: str
   - agent_model: str                        # e.g., "gemini-2.0-flash" or "claude-sonnet-4-6"
   - role: str                               # e.g., "proponent", "opponent",
"devil_advocate", "voter"
   - round_number: int
   - content: str                            # The actual argument/answer text
   - confidence: float                       # 0.0-1.0
```

```
   - predicted_group_answer: str | None       # For ISP voting — what this agent thinks others
will say
   - content_hash: str                        # SHA-256 of content
   - timestamp: datetime

5. VerifiedClaim — a claim locked by MAD-Oracle during debate:
   - claim_text: str
   - verified_by: str                         # Tool/method that verified it
   - round_locked: int
   - claim_hash: str

6. ConvergenceMetrics — tracked per round by the state monitor:
   - round_number: int
   - disagreement_entropy: float              # -Σ p(answer) * log(p(answer))
   - information_gain_delta: float            # |entropy_current - entropy_previous|
   - unique_answers: int
   - dominant_answer_share: float             # Fraction holding the most popular answer

7. DebateState — full state for the debate engine (LangGraph state schema):
   - task: str
   - task_features: TaskFeatures
   - round: int
   - max_rounds: int                          # Default 4
   - factions: dict[str, list[AgentOutput]]   # {"pro": [...], "opp": [...]}
   - cross_examinations: list[AgentOutput]
   - rebuttals: dict[str, list[AgentOutput]]
   - locked_claims: list[VerifiedClaim]
   - convergence_history: list[ConvergenceMetrics]
   - transcript_hashes: list[str]             # All content hashes (Merkle leaves)
   - final_answer: str | None
   - merkle_root: str | None
   - mechanism_switches: int                  # Count of mid-execution switches
   - terminated_early: bool

8. VoteState — full state for the vote engine (LangGraph state schema):
   - task: str
   - task_features: TaskFeatures
   - agent_outputs: list[AgentOutput]         # Each with answer + confidence +
predicted_group_answer
   - calibrated_confidences: dict[str, float] # agent_id → calibrated confidence
   - isp_scores: dict[str, float]             # answer → ISP surprise score
   - final_weights: dict[str, float]          # answer → aggregated weight
   - quorum_reached: bool
   - quorum_threshold: float                  # Default 0.6
   - final_answer: str | None
   - transcript_hashes: list[str]
   - merkle_root: str | None

9. DeliberationResult — unified output from any engine:
   - task: str
   - mechanism_used: MechanismType
   - mechanism_selection: MechanismSelection
   - final_answer: str
   - confidence: float
   - quorum_reached: bool
   - round_count: int
   - agent_count: int
   - mechanism_switches: int
   - merkle_root: str
   - transcript_hashes: list[str]
   - convergence_history: list[ConvergenceMetrics]
   - locked_claims: list[VerifiedClaim]       # Empty for vote engine
```

```
   - total_tokens_used: int
   - total_latency_ms: float
   - timestamp: datetime

10. BanditArm — internal state for Thompson Sampling:
    - mechanism: MechanismType
    - category: str                          # task category this arm applies to
    - alpha: float                           # Beta distribution α (successes + 1)
    - beta_param: float                      # Beta distribution β (failures + 1) — note:
can't name it 'beta' because of Pydantic conflicts
    - total_pulls: int
    - last_reward: float | None
"""
```

Important: Make sure every model has `model_config = ConfigDict(frozen=False)` where mutation is needed (states), and `frozen=True` for immutable records (AgentOutput, VerifiedClaim, etc.).

Task 2: Agent Caller Abstraction (`agora/agent.py`)

Build a unified async interface for calling LLM agents via the **Vertex AI SDK**. This is critical because:

- Different nodes in the graph need different models (Flash for generation, Pro/Sonnet for reasoning)
- The Vertex AI SDK gives us access to both Gemini and Claude models under one API
- We need to track token usage per call for benchmarking
- We need structured JSON output for several nodes

Important Vertex AI SDK notes:

- Gemini models are accessed via `langchain-google-vertexai` using `ChatVertexAI`
- Claude models on Vertex AI are accessed via `langchain-google-vertexai` using `ChatAnthropicVertex`
- Both support `with_structured_output()` for typed responses
- You need a GCP project with Vertex AI API enabled and `GOOGLE_CLOUD_PROJECT` env var set
- For Claude on Vertex, you also need the Anthropic model garden API enabled

```
"""
Implement class AgentCaller:

__init__(self, model: str = "gemini-2.0-flash", temperature: float = 0.7, project: str | None =
None, location: str = "us-central1"):
    Determine provider from model name:
      - If model starts with "gemini" → use ChatVertexAI from langchain_google_vertexai
      - If model starts with "claude" → use ChatAnthropicVertex from langchain_google_vertexai

    Initialize the appropriate LangChain chat model.
    Store model name, provider ("gemini" | "claude"), and project/location for logging.

    The project defaults to env var GOOGLE_CLOUD_PROJECT if not provided.

async call(
    self,
    system_prompt: str,
    user_prompt: str,
    response_format: type[BaseModel] | None = None,  # If provided, use with_structured_output
    temperature: float | None = None,                 # Override default
) -> tuple[str | BaseModel, dict]:
    \"""
    Returns (response_content, usage_metadata).
```

```
    usage_metadata = {"input_tokens": int, "output_tokens": int, "model": str, "latency_ms":
float, "provider": str}

    If response_format is provided, use model.with_structured_output(response_format)
    to get typed responses. Both ChatVertexAI and ChatAnthropicVertex support this.

    Handle retries (max 3) with exponential backoff for rate limits.
    Log every call via structlog with: model, provider, input_tokens, output_tokens,
latency_ms.

    NOTE on token counting:
    - Gemini models return usage_metadata on the AIMessage with usage_metadata dict
    - Claude on Vertex returns usage via response_metadata
    - Normalize both into the same output format
    \"""

Also create convenience constructors for the model tiers used across the system:

def flash_caller() -> AgentCaller:
    \"""Lightweight model for generation, voting, opening statements. Cost-efficient.\"""
    return AgentCaller(model="gemini-2.0-flash", temperature=0.7)

def pro_caller() -> AgentCaller:
    \"""Strong reasoning model for cross-examination, rebuttals, synthesis, mechanism
selection.\"""
    return AgentCaller(model="gemini-2.5-pro", temperature=0.5)

def claude_caller() -> AgentCaller:
    \"""Alternative strong reasoning model. Use when Gemini Pro is rate-limited or for
diversity.\"""
    return AgentCaller(model="claude-sonnet-4-6", temperature=0.5)

IMPORTANT: The convenience constructors define the model routing strategy for the whole system.
The rest of the codebase references flash_caller() and pro_caller() — NOT specific model
strings.
This means we can swap models (e.g., switch pro_caller to Claude) by changing ONE line.
"""
```

Task 3: Task Feature Extraction (`agora/selector/features.py`)

Extract features from a raw task string that the mechanism selector uses to decide which engine to route to.

```
"""
Implement async function extract_features(task_text: str, agent_count: int = 3, stakes: float =
0.5) -> TaskFeatures:

Feature extraction logic:

1. complexity_score (0.0-1.0):
   - Count tokens (approximate: len(task_text.split()) as proxy)
   - Count question marks, semicolons, nested clauses (words like "however", "although", "given
that")
   - Normalize: score = min(1.0, (token_count / 200) * 0.4 + (clause_indicators / 5) * 0.6)

2. topic_category:
   Use a simple keyword classifier (NOT an LLM call — too expensive for routing):
   - "math" if contains: calculate, equation, solve, integral, derivative, proof, theorem
   - "code" if contains: function, implement, debug, code, algorithm, python, javascript,
class, API
```

```
   - "reasoning" if contains: why, explain, analyze, compare, evaluate, argue, debate
   - "factual" if contains: what is, who is, when did, define, list, name
   - "creative" if contains: write, story, poem, design, imagine, create, brainstorm
   - Default: "reasoning"

3. expected_disagreement (0.0-1.0):
   Heuristic by category:
   - "math": 0.2 (usually one right answer)
   - "code": 0.4 (multiple valid approaches)
   - "factual": 0.15 (low disagreement)
   - "reasoning": 0.7 (high disagreement)
   - "creative": 0.9 (very high disagreement)

4. answer_space_size:
   - "math": 1-3
   - "factual": 1-5
   - "code": 3-10
   - "reasoning": 5-20
   - "creative": 50+

5. time_sensitivity: default from parameter (0.5 unless specified)

6. stakes: from parameter

7. agent_count: from parameter

Return TaskFeatures with all fields populated.
"""
```

Task 4: Thompson Sampling Bandit (`agora/selector/bandit.py`)

This is the core online-learning mechanism selector. It maintains Beta distribution posteriors for each (mechanism, task_category) pair and samples to select mechanisms.

```
"""
Implement class ThompsonSamplingSelector:

State: dict of BanditArm objects, keyed by (mechanism, category) tuples.

__init__(self, mechanisms: list[MechanismType] = ["debate", "vote"]):
    Initialize arms for each mechanism × each known category.
    Each arm starts with alpha=1.0, beta_param=1.0 (uniform prior).
    Known categories: ["math", "code", "reasoning", "factual", "creative"]

def select(self, features: TaskFeatures) -> tuple[MechanismType, float]:
    """
    Thompson Sampling selection.

    For each mechanism:
      sample = np.random.beta(arm.alpha, arm.beta_param)
      where arm is looked up by (mechanism, features.topic_category)

    Select mechanism with highest sample.
    Confidence = selected_sample / sum(all_samples)  # normalized

    Returns: (selected_mechanism, confidence)
    """
```

```
def update(self, mechanism: MechanismType, category: str, reward: float):
    """
    Update the arm's Beta posterior after observing a result.

    reward should be 0.0-1.0 (e.g., task accuracy, or binary 1.0 if correct).

    arm.alpha += reward
    arm.beta_param += (1.0 - reward)
    arm.total_pulls += 1
    arm.last_reward = reward
    """

def get_stats(self) -> dict:
    """Return all arm stats for logging/dashboard."""

def save_state(self, path: str):
    """Serialize arms to JSON for persistence across runs."""

def load_state(self, path: str):
    """Load arms from JSON."""

IMPORTANT IMPLEMENTATION NOTES:
- The bandit MUST work from run 1 with zero historical data. The uniform prior (α=1, β=1)
ensures
  random exploration initially, which is exactly what you want.
- After ~20-30 tasks per category, the posteriors will start to converge on the best mechanism.
- Thread-safety: use a simple lock if this will be called concurrently.
"""
```

Task 5: LLM Reasoning Agent Wrapper (`agora/selector/reasoning.py`)

Wraps the bandit with an LLM that provides chain-of-thought justification. This is what makes the selector *explainable* — the reasoning gets hashed and committed on-chain.

```
"""
Implement class ReasoningSelector:

__init__(self, caller: AgentCaller | None = None):
    Use pro_caller() by default — mechanism selection reasoning is high-stakes.

async def select(
    self,
    task_text: str,
    features: TaskFeatures,
    bandit_recommendation: tuple[MechanismType, float],
    historical_performance: dict | None = None,  # Optional stats from bandit
) -> MechanismSelection:
    """
    Prompt the LLM with:
    - The task text
    - Extracted features (as structured data)
    - The bandit's recommendation + confidence
    - Historical performance per mechanism (if available)

    System prompt (roughly):
    '''
    You are the Agora Mechanism Selector — a meta-reasoning agent that decides HOW a group
```

```
    of AI agents should collectively resolve a task. Your options are:

    - DEBATE: Structured adversarial deliberation with factional arguments and cross-
examination.
      Best for: complex reasoning, tasks with high stakes, tasks where there are clear opposing
      positions to evaluate, tasks requiring adversarial stress-testing.
    - VOTE: Independent parallel evaluation with confidence-weighted aggregation.
      Best for: simpler aggregation, factual questions, tasks where diversity of perspective
      matters more than depth of argumentation, time-sensitive tasks.
    - DELPHI: Anonymous iterative refinement. Best for: subjective questions, values-laden
      topics, tasks where conformity bias is a serious risk. (Available as extension)
    - MOA: Layered synthesis. Best for: creative/generative tasks where synthesis >
argumentation.
      (Available as extension)

    A statistical model has pre-analyzed this task and recommends: {bandit_recommendation}.
    Its confidence: {bandit_confidence}.

    Historical performance data: {historical_performance or "Not yet available — system is
still learning."}

    Your job: Consider the task, the features, and the statistical recommendation. You may
    agree or override the statistical model. Provide your selection and a detailed reasoning
    chain explaining WHY this mechanism is best for THIS specific task.

    Respond in JSON: {"mechanism": "debate"|"vote", "confidence": 0.0-1.0, "reasoning": "..."}
    '''

    Parse the response into MechanismSelection.
    Compute reasoning_hash = SHA-256 of the reasoning string.
    Include both the bandit's recommendation and the LLM's final decision in the output.
    """
```

Task 6: Combined Selector (`agora/selector/selector.py`)

Glue the bandit + reasoning agent into a single interface.

```
"""
Implement class AgoraSelector:

__init__(self, bandit_state_path: str | None = None):
    Create ThompsonSamplingSelector (load state if path provided).
    Create ReasoningSelector.
    Create feature extractor.

async def select(self, task_text: str, agent_count: int = 3, stakes: float = 0.5) ->
MechanismSelection:
    features = await extract_features(task_text, agent_count, stakes)
    bandit_rec = self.bandit.select(features)
    selection = await self.reasoning.select(task_text, features, bandit_rec,
self.bandit.get_stats())
    return selection

def update(self, selection: MechanismSelection, reward: float):
    self.bandit.update(selection.mechanism, selection.task_features.topic_category, reward)
"""
```

Task 7: Transcript Hasher + Merkle Tree Builder (`agora/runtime/hasher.py`)

Every piece of agent output gets hashed. All hashes form a Merkle tree. The root gets committed on-chain.

```
"""
Implement class TranscriptHasher:

Uses the `merkletools` library (pip install merkletools).

@staticmethod
def hash_content(content: str) -> str:
    """SHA-256 hash of content string. Returns hex digest."""

@staticmethod
def hash_agent_output(output: AgentOutput) -> str:
    """
    Deterministic hash of an agent output.
    Serialize to canonical JSON (sorted keys, no whitespace):
      {"agent_id": ..., "content": ..., "round_number": ..., "role": ..., "timestamp": ISO8601}
    Then SHA-256 the JSON string.
    IMPORTANT: The hash must be deterministic — same input always produces same hash.
    """

def build_merkle_tree(self, hashes: list[str]) -> str:
    """
    Build a Merkle tree from a list of content hashes.
    Returns the Merkle root as hex string.

    Use merkletools:
      mt = MerkleTools(hash_type="SHA256")
      mt.add_leaf(hashes, do_hash=False)  # Leaves are already hashed
      mt.make_tree()
      return mt.get_merkle_root()
    """

def build_receipt(
    self,
    hashes: list[str],
    mechanism: MechanismType,
    final_answer: str,
    quorum_reached: bool,
    round_count: int,
    mechanism_switches: int,
) -> dict:
    """
    Build a complete receipt for on-chain submission.
    Returns: {
        "merkle_root": str,
        "leaf_count": int,
        "mechanism": str,
        "final_answer_hash": str,  # Hash of the final answer
        "quorum_reached": bool,
        "round_count": int,
        "mechanism_switches": int,
        "timestamp": ISO8601 str,
    }
    """
```

Task 8: Mid-Execution State Monitor (`agora/runtime/monitor.py`)

Monitors convergence metrics during debate/vote execution and can trigger mechanism switches or early termination.

```
"""
Implement class StateMonitor:

Tracks convergence metrics across rounds and implements two key decisions:
1. Should the current mechanism terminate early? (information gain plateau)
2. Should we switch to a different mechanism? (divergence detected)

def compute_metrics(self, agent_outputs: list[AgentOutput]) -> ConvergenceMetrics:
    """
    Given all agent outputs from the current round:

    1. Count unique answers (normalize/lowercase for comparison).
    2. Compute answer distribution: p(answer) = count(answer) / total_agents.
    3. Compute disagreement_entropy = -Σ p * log2(p) for all unique answers.
       (Use log2 so entropy of a fair coin = 1.0)
    4. dominant_answer_share = max(p(answer)).
    """

def should_terminate(
    self,
    convergence_history: list[ConvergenceMetrics],
    min_rounds: int = 2,
    plateau_threshold: float = 0.05,
    plateau_rounds: int = 2,
) -> tuple[bool, str]:
    """
    Adaptive termination based on MACI information-gain plateau detection.

    Returns (should_terminate, reason).

    Logic:
    - Never terminate before min_rounds.
    - After min_rounds, check if information_gain_delta < plateau_threshold
      for the last plateau_rounds consecutive rounds.
    - If yes: terminate with reason "Information gain plateau detected".
    - Also terminate if dominant_answer_share > 0.9 (strong consensus reached).
    """

def should_switch_mechanism(
    self,
    convergence_history: list[ConvergenceMetrics],
    current_mechanism: MechanismType,
    min_rounds_before_switch: int = 2,
) -> tuple[bool, MechanismType | None, str]:
    """
    Detect when the current mechanism is failing and suggest a switch.

    Returns (should_switch, suggested_mechanism, reason).

    Logic:
    - If entropy is RISING for 2 consecutive rounds (agents getting MORE confused):
      → Switch debate to vote (cut losses, aggregate what we have)
      → Switch vote to debate (voting isn't converging, need adversarial reasoning)
    - If after max_rounds/2, dominant_answer_share < 0.3 (no answer gaining traction):
```

```
      → Suggest switching mechanisms.
    """
```

Task 9: Factional Debate Engine (`agora/engines/debate.py`)

This is the most complex component. It's a LangGraph StateGraph implementing factional adversarial debate with adaptive termination.

```
"""
Implement the debate engine as a LangGraph StateGraph.

The graph has these nodes:

1. "assign_factions" — Entry node
   - Take the task and N agents
   - Have each agent generate an initial answer independently (parallel, use Send API)
   - Group agents into factions based on their answers:
     - Find the top-2 most common initial answers
     - Agents who chose answer A → "pro" faction (argue for A)
     - Agents who chose answer B → "opp" faction (argue for B)
     - If there's a clear outlier → assign as Devil's Advocate
     - If no natural DA, designate one agent (preferably from the minority faction) as DA
   - Hash all initial outputs and add to transcript_hashes

2. "opening_statements" — Parallel node
   - Each faction agent generates a structured opening argument for their assigned position
   - System prompt for proponents:
     "You are arguing IN FAVOR of the answer: '{faction_answer}'.
      Present your strongest evidence and reasoning. Be specific and cite concrete logic.
      Structure your argument as: {claim: str, evidence: str, confidence: float}"
   - Same structure for opponents with the opposing answer
   - Hash all outputs → transcript_hashes

3. "cross_examination" — Devil's Advocate node
   - DA receives all opening statements from both factions
   - System prompt:
     "You are the Devil's Advocate. Your ONLY job is to find flaws.
      For EACH faction's opening statement, identify:
      1. The weakest claim and why it fails
      2. Any logical gaps, unsupported assertions, or contradictions
      3. A targeted question that would expose the weakness

      Be ruthless. Do NOT agree with either side. Do NOT offer your own answer.
      Output for each faction: {faction: str, weakest_claim: str, flaw: str, question: str}"
   - Hash output → transcript_hashes

4. "rebuttal" — Parallel node
   - Each faction receives the DA's cross-examination targeting them
   - Must defend their position against the specific challenges raised
   - System prompt includes any locked_claims:
     "The following claims have been VERIFIED and are locked — do not attack them:
{locked_claims}"
   - Check for verifiable claims:
     - If any agent makes a mathematical claim → attempt verification (e.g., simple eval)
     - If verified → add to locked_claims as VerifiedClaim
   - Hash all outputs → transcript_hashes

5. "convergence_check" — Conditional edge node
```

```
   - Use StateMonitor.compute_metrics() on the latest round's outputs
   - Use StateMonitor.should_terminate() → if true, go to "final_aggregation"
   - Use StateMonitor.should_switch_mechanism() → if true, set mechanism_switches += 1
     and route to vote engine (return special state flag)
   - Otherwise → loop back to "cross_examination" (next round)

6. "final_aggregation" — Terminal node
   - Collect ALL outputs across ALL rounds (trajectory-aware scoring)
   - Score each faction's cumulative argument strength:
     - Weight by recency (later rounds > earlier rounds): weight = 1.0 + 0.2 * round_number
     - Weight by confidence: multiply by agent's stated confidence
     - Bonus for having locked verified claims
   - Determine winning faction
   - Generate final answer (use pro_caller to synthesize the winning argument)
   - Build Merkle tree from all transcript_hashes → merkle_root
   - Return DeliberationResult

GRAPH STRUCTURE:
  START → assign_factions → opening_statements → cross_examination → rebuttal →
convergence_check
                                                                  ↑                                  │
                                                                  └──── (if not converged) ──────────┘
                                                                                                     │
                                                                                (if converged) ──────→
final_aggregation → END
                                                                                (if switch) ─────────→
SWITCH_TO_VOTE (special return)

IMPLEMENTATION NOTES:
- Use LangGraph's StateGraph with DebateState as the state schema.
- For parallel fan-out (opening_statements, rebuttal), use the Send API to dispatch to
  N agent nodes in parallel, with a custom reducer that collects results into lists.
- The cyclic subgraph (cross_exam → rebuttal → convergence → cross_exam) uses
conditional_edges.
- Use flash_caller() for opening statements and rebuttals (cost-efficient generation).
- Use pro_caller() for cross-examination and final aggregation (needs stronger reasoning).
- Track total_tokens_used across all LLM calls.
- Stream agent outputs via LangGraph's built-in streaming (for the dashboard later).

TARGET: Each debate should complete in 2-4 rounds with 3-5 agents in under 60 seconds.
"""
```

Task 10: ISP-Weighted Vote Engine (`agora/engines/vote.py`)

Simpler than debate but with the novel ISP (Inverse Surprising Popularity) aggregation.

```
"""
Implement the vote engine as a LangGraph StateGraph.

The graph has these nodes:

1. "generate_votes" — Parallel fan-out node
   - Each agent independently generates:
     {
       answer: str,
       confidence: float (0.0-1.0),
       predicted_group_answer: str,  # KEY ISP FIELD: "What do you think most other agents will
answer?"
       reasoning: str
```

```
     }
   - System prompt:
     "Answer the following task. Provide:
      1. Your answer
      2. Your confidence (0.0-1.0) in your answer
      3. Your prediction: what do you think MOST other agents will answer? (This may differ
from your own answer.)
      4. Brief reasoning

      The prediction question is important — think carefully about what others are likely to
say,
      which may be different from what you believe is correct."
   - Hash all outputs → transcript_hashes

2. "calibrate_confidence" — Processing node
   - Apply temperature scaling to raw confidences:
     calibrated = sigmoid(logit(raw_confidence) / T)
     where T is a temperature parameter (default T=1.5 — softens overconfident predictions)
     logit(p) = log(p / (1-p))
   - Store calibrated_confidences in state

3. "isp_aggregate" — Processing node
   - For each unique answer A:
     actual_frequency(A) = count of agents who chose A / total_agents
     predicted_frequency(A) = mean of all agents' predicted_group_answer matching A /
total_agents
     surprise_score(A) = actual_frequency(A) - predicted_frequency(A)
   - For each unique answer A:
     weight(A) = Σ calibrated_confidence[agent] * (1 + surprise_score(A))
       for all agents who chose answer A
   - Normalize weights to sum to 1.0
   - Store isp_scores and final_weights in state

4. "quorum_check" — Conditional edge node
   - If max(final_weights) > quorum_threshold → quorum reached
   - If quorum reached → go to "finalize"
   - If NOT reached → flag for potential mechanism switch (to debate)

5. "finalize" — Terminal node
   - Set final_answer = answer with highest weight
   - Build Merkle tree from all transcript_hashes
   - Return DeliberationResult

GRAPH STRUCTURE:
  START → generate_votes → calibrate_confidence → isp_aggregate → quorum_check → finalize → END

IMPLEMENTATION NOTES:
- This is much simpler than debate — no cycles, no multiple rounds.
- Use flash_caller() for all vote generation (cost-efficient).
- The ISP aggregation is pure math — no LLM calls needed.
- The predicted_group_answer field is what makes this novel. Without it, you're just doing
  confidence-weighted majority vote. WITH it, you get surprising-popularity boosting.
- Handle edge case: if all agents predict the same group answer, surprise_score = 0 for
everything,
  and you fall back to pure confidence-weighted voting. This is fine.

TARGET: Vote should complete in under 10 seconds for 5 agents (single parallel LLM call).
"""
```

Task 11: Main Orchestrator (`agora/runtime/orchestrator.py`)

Ties everything together into a single async pipeline.

```
"""
Implement class AgoraOrchestrator:

__init__(
    self,
    agent_count: int = 3,
    bandit_state_path: str | None = None,
    default_stakes: float = 0.5,
):
    Create AgoraSelector, DebateEngine, VoteEngine, TranscriptHasher, StateMonitor.

async def run(self, task: str, stakes: float | None = None) -> DeliberationResult:
    """
    Full orchestration pipeline:

    1. Feature extraction
    2. Mechanism selection (bandit + LLM reasoning)
    3. Log: mechanism selected, confidence, reasoning hash
    4. Execute selected engine
       - If debate returns SWITCH_TO_VOTE flag → run vote engine on same task
       - Increment mechanism_switches
    5. Build Merkle tree receipt
    6. Return DeliberationResult
    7. (On-chain submission is handled by the API layer — not here)
    """

async def run_and_learn(self, task: str, ground_truth: str | None = None) ->
DeliberationResult:
    """
    Run + update bandit with reward signal.

    If ground_truth provided: reward = 1.0 if final_answer matches, 0.0 otherwise.
    If no ground_truth: reward = result.confidence * (1.0 if result.quorum_reached else 0.5)
    """
```

Task 12: Tests

Write tests for each component. Use pytest + pytest-asyncio.

```
"""
tests/test_bandit.py:
- Test that Thompson Sampling with uniform priors produces roughly equal selection rates for 2
arms over 100 trials.
- Test that after 20 updates with reward=1.0 for "debate" and reward=0.0 for "vote" on category
"reasoning",
  the bandit selects "debate" >80% of the time.
- Test save/load state roundtrip.

tests/test_hasher.py:
- Test that hash_content is deterministic (same input → same hash).
- Test that hash_agent_output produces different hashes for different content.
- Test that build_merkle_tree with known inputs produces expected root.
- Test that build_receipt contains all required fields.
```

```
tests/test_debate.py:
- Test that assign_factions creates exactly 2 factions + optional DA.
- Test full debate run with 3 mock agents on a simple math question.
- Test that adaptive termination fires when convergence plateaus.
- Test that mechanism switch triggers when entropy rises.

tests/test_vote.py:
- Test ISP aggregation with known inputs:
  - 3 agents all agree → weight should be high, quorum reached.
  - 2 agents agree, 1 disagrees, and the disagreeing agent predicted others would agree →
    the majority answer gets surprise-boosted.
- Test confidence calibration reduces extreme values.
- Test quorum check with threshold = 0.6.

tests/test_orchestrator.py:
- Test full pipeline: task → selection → execution → result with all fields populated.
- Test mechanism switch: provide a task where debate should fail → verify switch to vote
occurs.
"""
```

Execution Order

Implement in this exact order — each task depends on the ones before it:

1. **Task 0** — Repo structure + pyproject.toml
2. **Task 1** — types.py (everything imports from here)
3. **Task 2** — agent.py (LLM abstraction used everywhere)
4. **Task 3** — features.py (needed by selector)
5. **Task 4** — bandit.py (core selector logic)
6. **Task 5** — reasoning.py (LLM wrapper for selector)
7. **Task 6** — selector.py (glue)
8. **Task 7** — hasher.py (needed by both engines)
9. **Task 8** — monitor.py (needed by debate engine)
10. **Task 9** — debate.py (most complex component)
11. **Task 10** — vote.py (simpler engine)
12. **Task 11** — orchestrator.py (ties it all together)
13. **Task 12** — Tests

After completing all tasks, run the full test suite and fix any issues. Then run a manual end-to-end test:

```python
import asyncio
from agora.runtime.orchestrator import AgoraOrchestrator

async def main():
    orch = AgoraOrchestrator(agent_count=3)

    # Test with different task types
    tasks = [
        "What is the derivative of x^3 * sin(x)?",  # Should likely route to debate
        "What is the capital of France?",              # Should likely route to vote
        "Compare the trade-offs between microservices and monolithic architectures for a
startup with 5 engineers.",  # Complex reasoning
    ]

    for task in tasks:
        result = await orch.run(task)
```

```
        print(f"\nTask: {task}")
        print(f"Mechanism: {result.mechanism_used}")
        print(f"Answer: {result.final_answer}")
        print(f"Confidence: {result.confidence}")
        print(f"Quorum: {result.quorum_reached}")
        print(f"Rounds: {result.round_count}")
        print(f"Switches: {result.mechanism_switches}")
        print(f"Merkle Root: {result.merkle_root}")

asyncio.run(main())
```

Quality Checklist

Before moving to Phase 2, verify:

- [ ] All types defined and importable from `agora.types`
- [ ] Thompson Sampling produces sensible selections with zero prior data
- [ ] Feature extraction correctly categorizes diverse task types
- [ ] LLM reasoning agent provides coherent mechanism selection justification
- [ ] Debate engine completes in <60s with 3 agents, terminates adaptively
- [ ] Vote engine completes in <15s with 5 agents
- [ ] ISP aggregation produces different results than simple majority vote
- [ ] Merkle tree construction is deterministic
- [ ] Mid-execution state monitor correctly detects convergence plateaus
- [ ] Full orchestrator pipeline runs end-to-end without errors
- [ ] All tests pass
- [ ] No bare `except` clauses, all errors handled with specific exception types
- [ ] Structured logging shows clear execution trace